ADVENTIST UNIVERSITY OF CENTRAL AFRICA

Name: Uwayezu Bienvenu
Student ID: 101362

AUCA | ANALYTICS LAB

Distributed Multi-Model Analytics for E-commerce Data

Technical project report with evidence screenshots

MongoDB | Apache HBase | Apache Spark

Contents

Summary of results 4

1. Architecture and data 5

2. MongoDB and HBase design 8

3. Spark processing and integration 15

4. Visual findings 20

5. Interactive dashboard 25

6. Scalability, limitations and reproduction 31

Appendix A. Evidence timeline 32

Appendix B. Environment and versions 32

Appendix C. Data dictionary 33

Appendix D. Claim-to-evidence matrix 35

Appendix E. Reconciliation of figures across sources 36

Appendix F. Glossary 37

List of figures

Figure 1. Full-scale generator run . . . 6
Figure 2. Dataset manifest . . . 7
Figure 3. MongoDB Compass: categories . . . 9
Figure 4. MongoDB Compass: products . . . 10
Figure 5. HBase loader output . . . 12
Figure 6. HBase Master status . . . 13
Figure 7. HBase Region Visualizer . . . 14
Figure 8. HBase software attributes . . . 15
Figure 9. Spark UI during the full-scale run . . . 17
Figure 10. Full-scale summary file . . . 18
Figure 11. Fulfilled revenue by category . . . 21
Figure 12. Customer purchase frequency . . . 22
Figure 13. Top selling products . . . 23
Figure 14. Browsing and purchase funnel . . . 24
Figure 15. Dashboard: Overview . . . 26
Figure 16. Dashboard: Sales explorer . . . 27
Figure 17. Dashboard: Customer 360 . . . 28
Figure 18. Dashboard: HBase activity . . . 29
Figure 19. Dashboard: Data explorer . . . 30

Summary of results

This report documents a multi-model analytics pipeline for synthetic e-commerce data. MongoDB stores bounded business entities, HBase stores time-ordered page events, and Spark performs batch, cross-store and simulated-stream analysis. Every claim about a run is supported by a figure; Appendix D maps each claim to the figure that evidences it.

Project scope	Implemented evidence
MongoDB documents and analytics	Loaded into local MongoDB; both aggregation pipelines executed and results saved.
HBase wide-column model	Started local HBase 2.6.6 and Thrift; created tables and loaded 99,873 page-view rows.
Spark processing and integration	Processed all 2,000,000 sessions and 500,000 transactions with Spark summaries; separately joined MongoDB and HBase sources on the assignment sample.
Visual analytics	Four static charts from the assignment-scale sample; a synthetic Spark stream also produced windowed microbatch aggregates. An interactive dashboard over MongoDB and HBase (Section 5) is documented with screenshots.

Scale distinction: the full generator produced 10,000 users, 5,000 products, 25 categories, 2,000,000 sessions and 500,000 transactions, and Spark batch analysis ran over those files. MongoDB and HBase plus the direct cross-store integration use the manageable assignment sample: 1,000 users, 500 products, 10,000 sessions and 2,500 transactions.

Headline measures

Measure	Full-scale Spark batch	Assignment-scale sample
Users	10,000	1,000
Products	5,000	500
Sessions	2,000,000	10,000
Transactions	500,000 generated; 439,067 fulfilled	2,500 generated
Converted sessions	405,887 (20.3%)	2,057 (20.6%)
Fulfilled revenue	399,384,245.22	Leading category cat_009: 274,758.91 item revenue
Event rows in HBase	not loaded (sample only)	99,873 page views; 24,881 product/day rows

How this document is organised

Sections 1 to 3 describe the architecture, the MongoDB and HBase designs and the Spark processing. Each run is evidenced on its own page: a full-width screenshot, the values read from it, and a note on how it supports the report. Section 4 presents the four generated charts with a short reading of each. Section 5 documents the interactive dashboard. Section 6 covers scalability, limitations and reproduction. The appendices give an evidence timeline, environment versions, a data dictionary, a claim-to-figure matrix, a reconciliation of figures across sources, and a glossary.

1. Architecture and data

The design assigns each workload to the storage or processing model that best fits its access pattern. MongoDB holds catalog, profile, session-summary and transaction documents. HBase stores high-volume page-view events for keyed time-range retrieval. Spark batch analytics reads the generated JSON files. A separate integration mode reads profiles and fulfilled transactions from MongoDB through PyMongo and event rows from HBase through HappyBase, stages those extracts as JSON, then joins and aggregates them with Spark DataFrames. The store integration uses the assignment-scale load; full-scale batch processing uses the generated full dataset.

Stage	Technology	Purpose
Source	JSON files	Synthetic users, products, categories, sessions and transactions.
Operational documents	MongoDB	Nested records, selective indexes, lookup and group pipelines.
Event retrieval	HBase	Sparse page events keyed for a user-first recent-activity scan.
Analytics	Spark	Distributed joins, cohort aggregation, co-purchase and integrated metrics.
Presentation	Matplotlib + PDF	Four static plots and findings for a reviewable report.

Data preparation

The supplied dataset_generator.py is an exported notebook JSON object, not an executable Python module. run_assignment_generator.py extracts its code cell, overrides the scale values, and runs that code unchanged. The assignment-scale store sample contains 1,000 users, 500 products, 10,000 sessions and 2,500 transactions. The same runner generated the brief defaults: 10,000 users, 5,000 products, 25 categories, 500,000 transactions and 2,000,000 sessions, written as bounded chunks. Spark batch analysis was executed on the full files; MongoDB and HBase intentionally retain the representative sample. A separate standard-library sample generator remains available for quick smoke runs.

Pipeline sequence

Generate JSON -> MongoDB and HBase loaders (optional) -> Spark normalization and analysis -> Parquet/CSV aggregates plus PNG charts -> report. On a cluster, keep joins distributed, partition on user or event date where appropriate, and write compact columnar outputs for repeat analysis.

The next two pages show the full-scale generation run and the manifest it wrote.

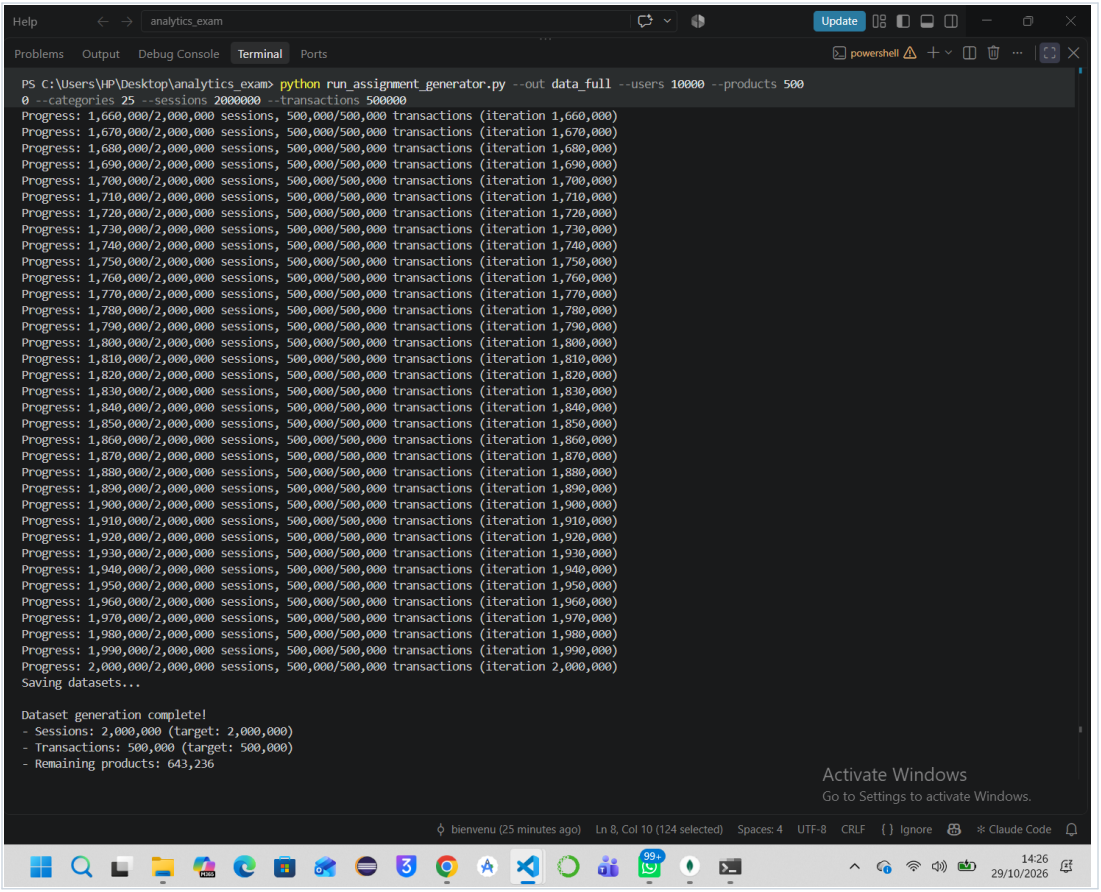


Figure 1. Full-scale generator run in the VS Code terminal (29/10/2026, 14:26).

Values read from the screenshot

Item	Observed
Command	`python run_assignment_generator.py --out data_full --users 10000 --products 5000 --categories 25 --sessions 2000000 --transactions 500000`
Progress output	One line per 10,000 iterations, e.g. "1,660,000/2,000,000 sessions, 500,000/500,000 transactions"
Transactions	Already at 500,000/500,000 in every visible progress line; only the session count still advances
Final lines	"Saving datasets..." then "Dataset generation complete!"
Sessions	2,000,000 (target: 2,000,000)
Transactions	500,000 (target: 500,000)
Remaining products	643,236 (printed by the generator; not interpreted in this report)

Relevance to the report

This is the evidence that the full dataset in Section 1 was generated with the brief defaults, using the unchanged generator code through its runner. The run reached both targets exactly.

Reading notes

Progress is reported by iteration. The "Remaining products" figure is part of the generator output and has no bearing on the Spark results reported later.

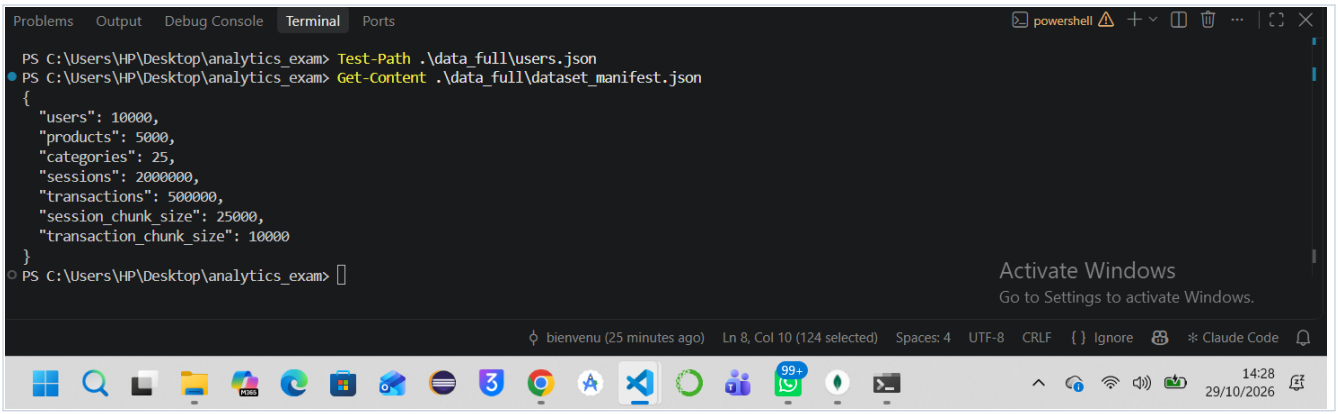


Figure 2. Contents of data_full\dataset_manifest.json (29/10/2026, 14:28).

Values read from the screenshot

Item	Observed
Command	`Get-Content .\data_full\dataset_manifest.json`
users	10,000
products	5,000
categories	25
sessions	2,000,000
transactions	500,000
session_chunk_size	25,000
transaction_chunk_size	10,000

Relevance to the report

The manifest documents the scale of the full dataset and its chunking. The chunk sizes fix the number of files Spark must read: 2,000,000 / 25,000 = 80 session files and 500,000 / 10,000 = 50 transaction files, which matches the path counts in the Spark UI (Figure 9).

Reading notes

A Test-Path check on users.json was run first in the same terminal. Its result is not visible in the capture, so the manifest is used as the evidence that the output folder was written.

2. MongoDB and HBase design

MongoDB collections

Collection	Document boundary and index rationale
products	One document per product. Keep price history bounded and category/subcategory IDs for joins. Index product_id uniquely and category_id + is_active for catalog filtering.
users	One document per user with nested geo and registration fields. Keep large session histories out of this document. Index user_id uniquely.
transactions	One document per order, with its bounded items array embedded. Index transaction_id, timestamp + status, and user_id + timestamp.
sessions	One document per session with nested page_views, device and cart. This keeps the journey readable as one unit. Index user_id + start_time descending.

The implemented product-popularity pipeline filters fulfilled orders, unwinds items, groups units and item revenue, looks up product names and ranks the top 20. The category-revenue pipeline unwinds items, joins products and groups revenue, units and distinct orders by category. Use allowDiskUse for larger group stages. For repeatable imports, productionize replacement with bulk upserts and validate duplicate identifiers before loading.

The loaded database auca_ecommerce contains the categories, products, sessions, transactions and users collections, as inspected in MongoDB Compass on the following two pages.

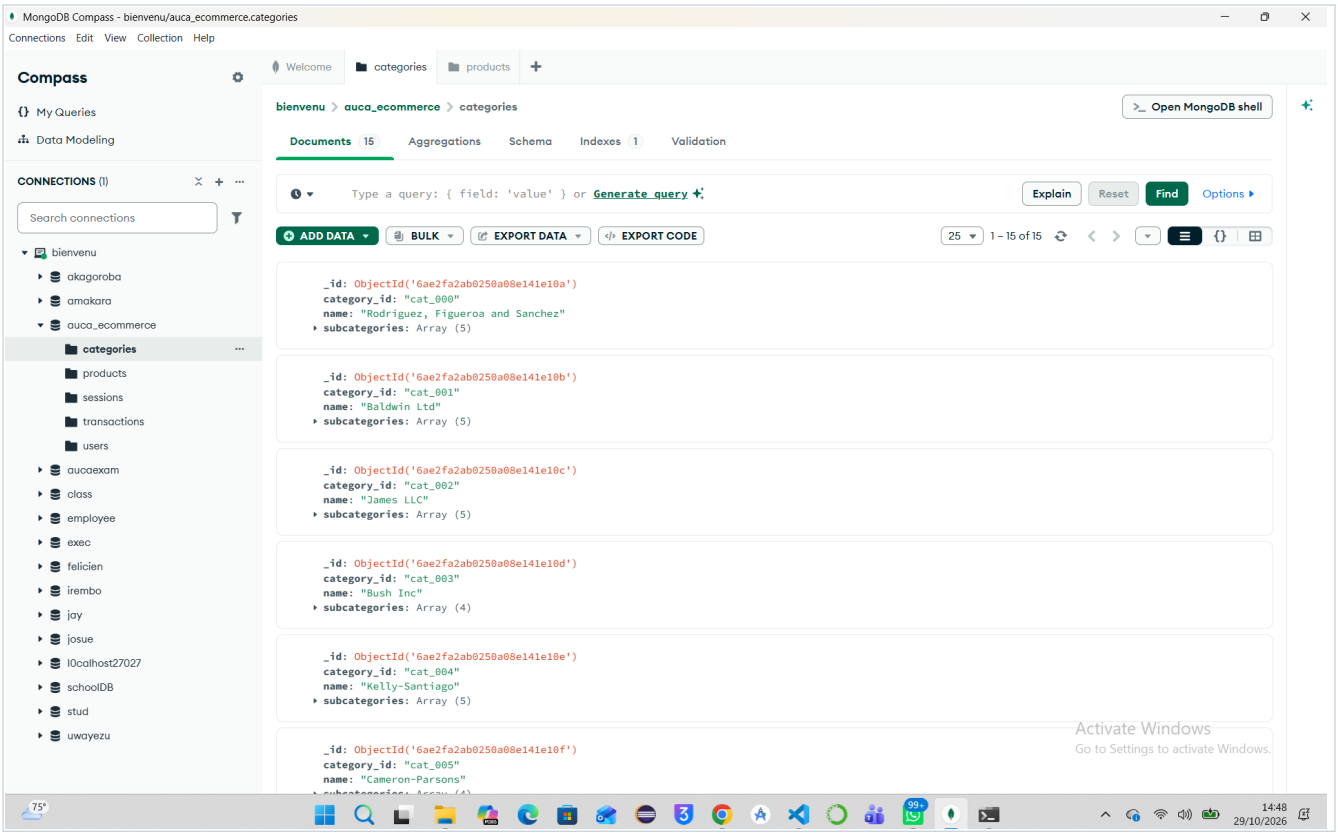


Figure 3. MongoDB Compass: auca_ecommerce.categories (29/10/2026, 14:48).

Values read from the screenshot

Item	Observed
Connection / database	bienvenu > auca_ecommerce > categories
Collections in database	categories, products, sessions, transactions, users
Documents / indexes	15 documents; 1 index (the default _id index)
cat_000	"Rodriguez, Figueroa and Sanchez"; subcategories: Array (5)
cat_001	"Baldwin Ltd"; subcategories: Array (5)
cat_002	"James LLC"; subcategories: Array (5)
cat_003	"Bush Inc"; subcategories: Array (4)
cat_004	"Kelly-Santiago"; subcategories: Array (5)
cat_005	"Cameron-Parsons"; subcategories array (partly cut off)

Relevance to the report

The categories collection shows the document model described above: one document per category with its subcategories embedded as a bounded array. Its 15 documents match the 15 categories visible in the assignment-scale charts (Figure 11).

Reading notes

The full-scale manifest lists 25 categories, whereas this MongoDB collection holds the 15 of the assignment sample, consistent with the report's scale distinction. Category names are synthetic company-style strings.

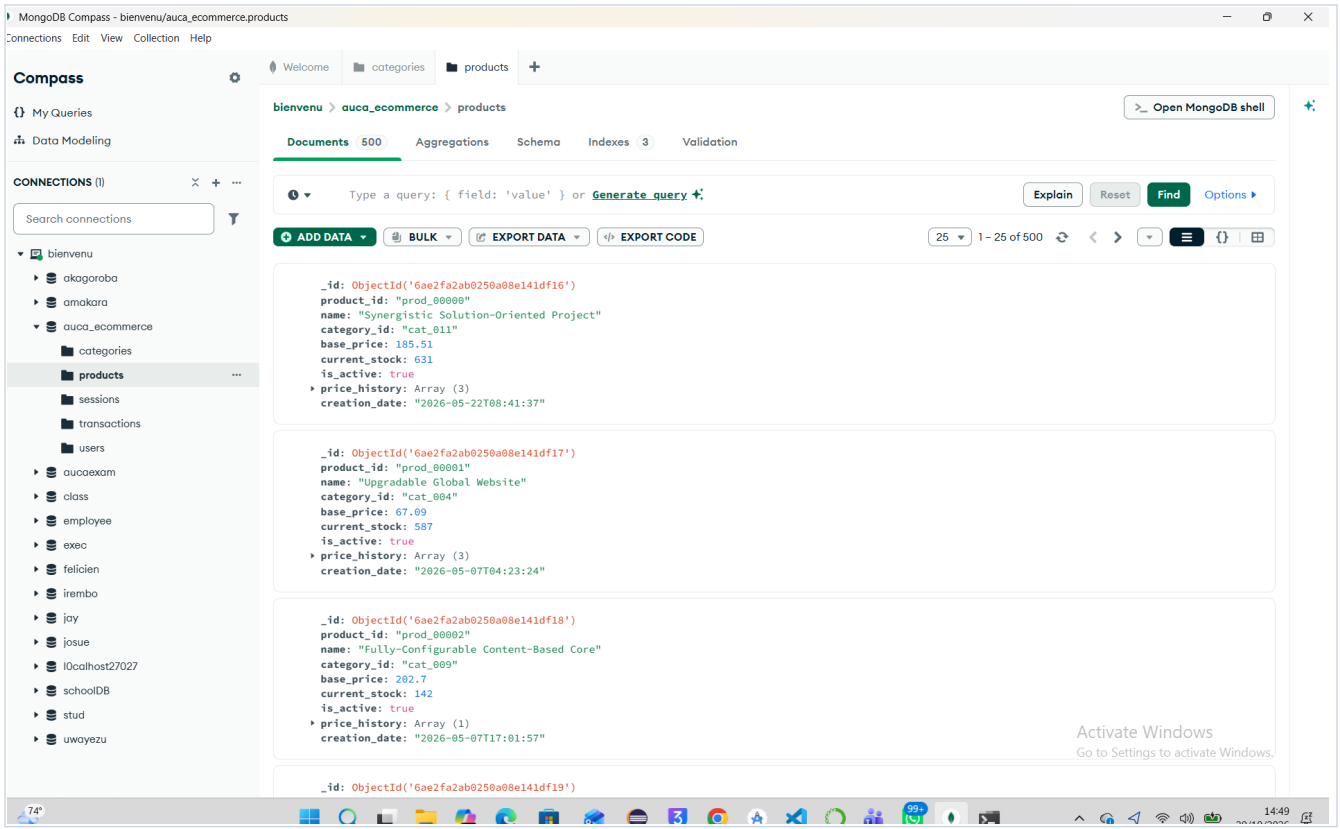


Figure 4. MongoDB Compass: auca_ecommerce.products (29/10/2026, 14:49).

Values read from the screenshot

Item	Observed
Documents / indexes	500 documents (shown 1 to 25 of 500); 3 indexes
Fields per document	_id, product_id, name, category_id, base_price, current_stock, is_active, price_history, creation_date
prod_00000	"Synergistic Solution-Oriented Project"; cat_011; base_price 185.51; current_stock 631; price_history Array (3)
prod_00001	"Upgradable Global Website"; cat_004; base_price 67.09; current_stock 587; price_history Array (3)
prod_00002	"Fully-Configurable Content-Based Core"; cat_009; base_price 202.7; current_stock 142; price_history Array (1)
is_active	true on all visible documents
creation_date	ISO-like strings, e.g. "2026-05-22T08:41:37"

Relevance to the report

The products collection confirms the 500-product assignment sample and the design choices in the collections table: category_id for joins, a bounded price_history array, and a stock snapshot in current_stock. Three indexes are consistent with the unique product_id index, the category_id + is_active index and the default _id index.

Reading notes

The report describes inventory values as snapshots rather than a stock ledger; the single current_stock field per product is what that limitation refers to. The index definitions themselves are not visible here, so the mapping of the three indexes to those names is an inference from the design.

HBase tables

Table	Row key	Column families
user_events	user_id | reverse_epoch_ms | session_id | event_index	s: session attributes; e: page type, product, category, duration, event timestamp
product_daily	product_id | yyyyMMdd	m: product views, cart-add units, purchased units, revenue in cents

The user_events key clusters event rows for each user and sorts newest first because the epoch is reversed. It supports a contiguous row-key range scan for one user. The product_daily table supports a product and date range query. Prefixing user IDs in a globally ordered table can hotspot a small number of users; salt or region partitioning can spread write load, at the cost of fan-out reads. Avoid large session blobs in a single HBase cell.

HBase execution

The local standalone HBase 2.6.6 service and Thrift gateway ran on ports 16010 and 9090. The loader processed 10,000 sessions for 1,000 users and wrote 99,873 page-view rows to user_events; a subsequent table scan returned the same 99,873 rows. The loader stores session metadata in family s and event attributes in family e. It also loaded 24,881 product/day rows. The user-prefix query for user_000042 returned 20 event rows, confirming the reverse-time key scan against a concrete user.

The following four pages show the loader output, the HBase Master status page, the region storage chart and the software attributes.

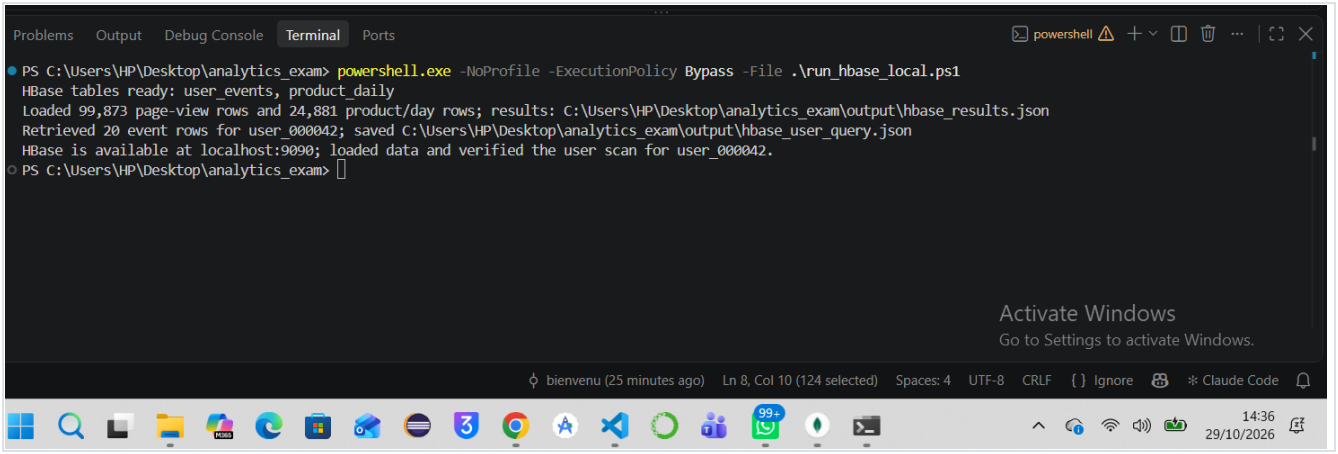


Figure 5. Output of run_hbase_local.ps1 in the VS Code terminal (29/10/2026, 14:36).

Values read from the screenshot

Item	Observed
Command	`powershell.exe -NoProfile -ExecutionPolicy Bypass -File .\run_hbase_local.ps1`
Tables	"HBase tables ready: user_events, product_daily"
Loaded	99,873 page-view rows and 24,881 product/day rows
Results file	output\hbase_results.json
User scan	20 event rows for user_000042, saved to output\hbase_user_query.json
Closing line	"HBase is available at localhost:9090; loaded data and verified the user scan for user_000042."

Relevance to the report

This capture is the primary evidence for the HBase execution paragraph above: the two tables exist, 99,873 and 24,881 rows were loaded, and the user-prefix scan for user_000042 returned 20 rows.

Reading notes

The row counts printed here equal the figures quoted in the Limitations section, and the 99,873 page views are those of the 10,000-session assignment sample, not the 2,000,000-session full dataset.

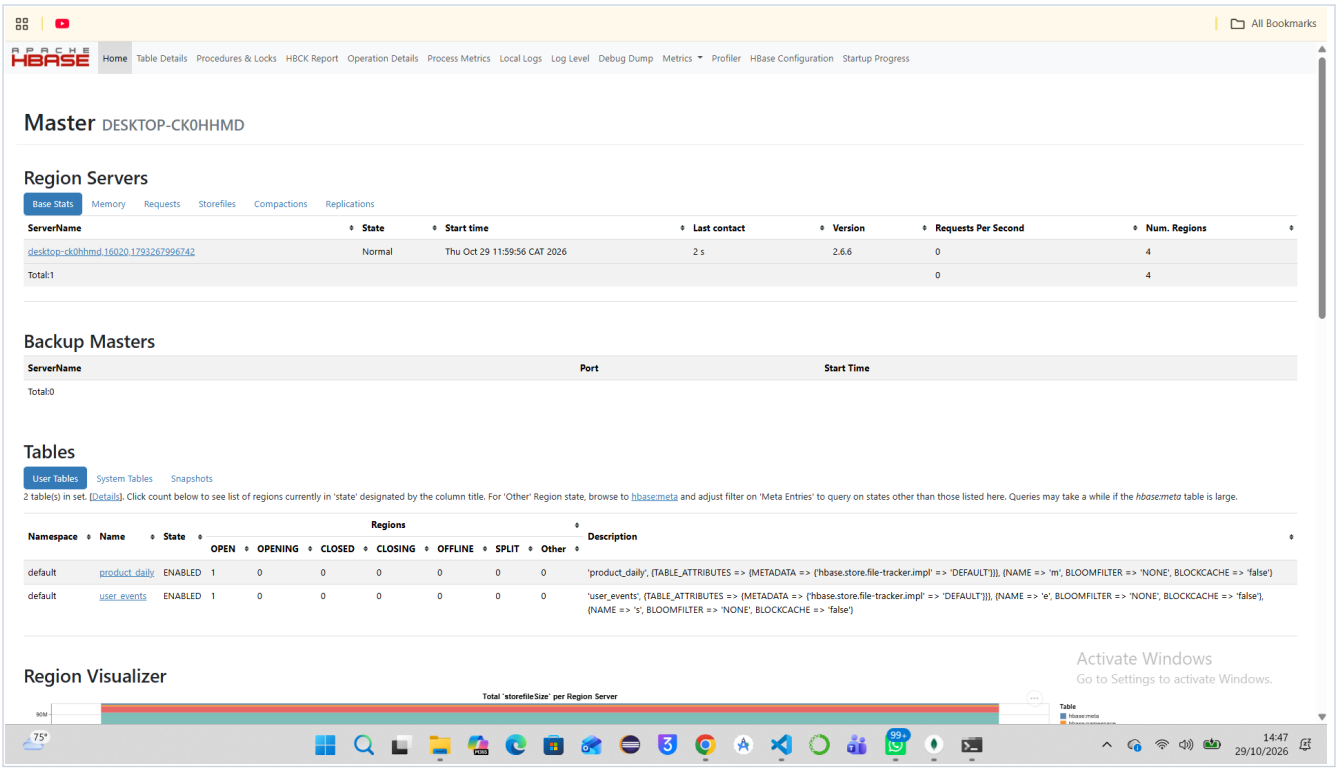


Figure 6. HBase Master web UI, home page (localhost:16010, 29/10/2026, 14:47).

Values read from the screenshot

Item	Observed
Master	DESKTOP-CK0HHMD
Region server	desktop-ck0hhmd,16020,... ; State Normal; Start time Thu Oct 29 11:59:56 CAT 2026
Version	2.6.6
Requests per second	0
Num. regions	4 (total, one region server)
Backup masters	Total: 0
User tables	product_daily (ENABLED, 1 open region) and user_events (ENABLED, 1 open region)
product_daily family	m; BLOOMFILTER NONE; BLOCKCACHE false
user_events families	e and s; BLOOMFILTER NONE; BLOCKCACHE false

Relevance to the report

The status page independently confirms the table design in the HBase tables section: user_events carries families e and s, and product_daily carries family m. It also confirms the single-node standalone topology and the 2.6.6 version.

Reading notes

Four regions are reported in total: one for each of the two user tables plus the hbase:meta and hbase:namespace system tables. The requests-per-second value is an instantaneous reading taken when the page was captured.

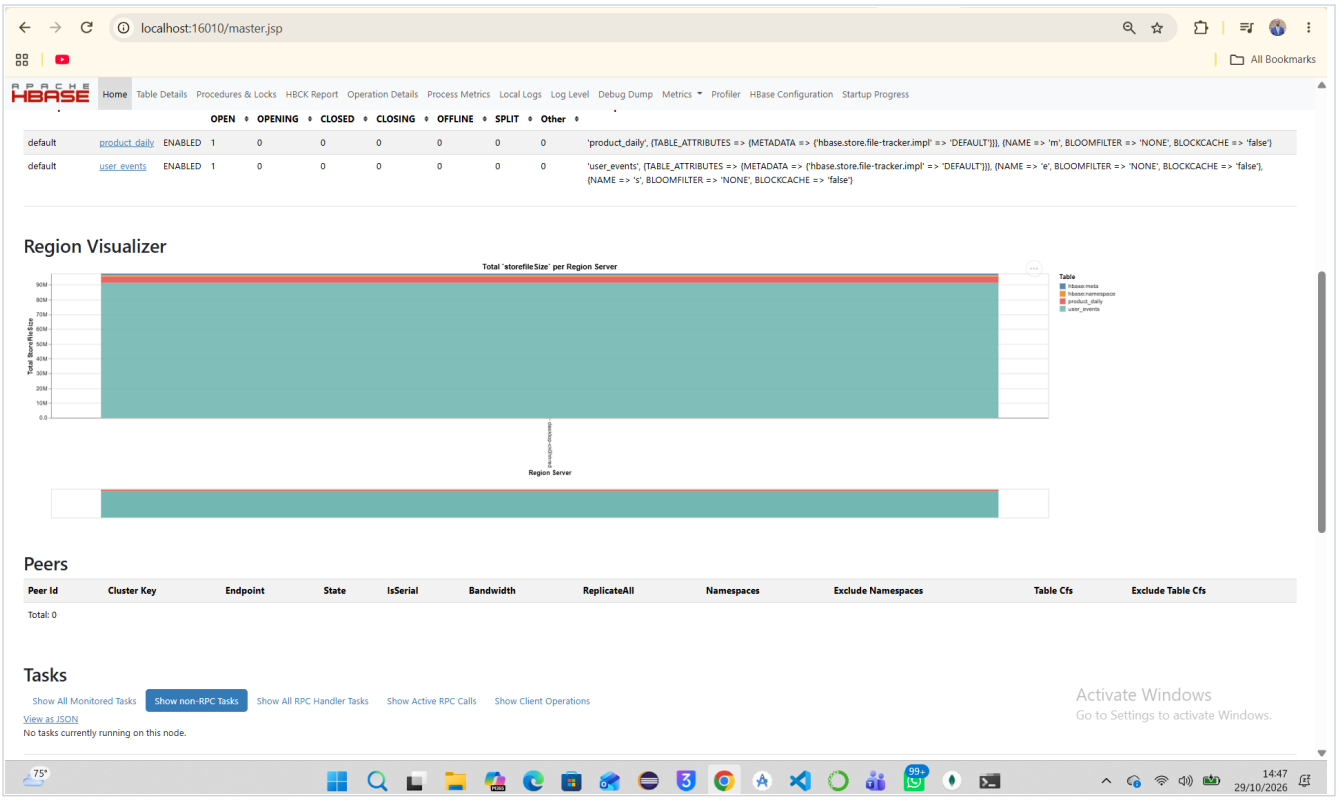


Figure 7. HBase Region Visualizer, total storefile size per region server (29/10/2026, 14:47).

Values read from the screenshot

Item	Observed
Chart	Total storefileSize per Region Server, stacked by table
Legend	hbase:meta, hbase:namespace, product_daily, user_events
Axis scale	0 to about 90M on the vertical axis
Dominant table	user_events fills almost the whole stack
Thin bands	product_daily and the system tables form narrow bands on top
Peers	Total: 0 (no replication peers)
Tasks	"No tasks currently running on this node."

Relevance to the report

The visualiser shows where the stored bytes sit. The page-event table user_events dwarfs the product/day table, which agrees with the row counts (99,873 event rows against 24,881 product/day rows) and with the wide-row event design.

Reading notes

The chart is a single-server view. It says nothing about distribution across servers, so it cannot be used to judge hotspotting; that analysis would need a multi-node cluster.

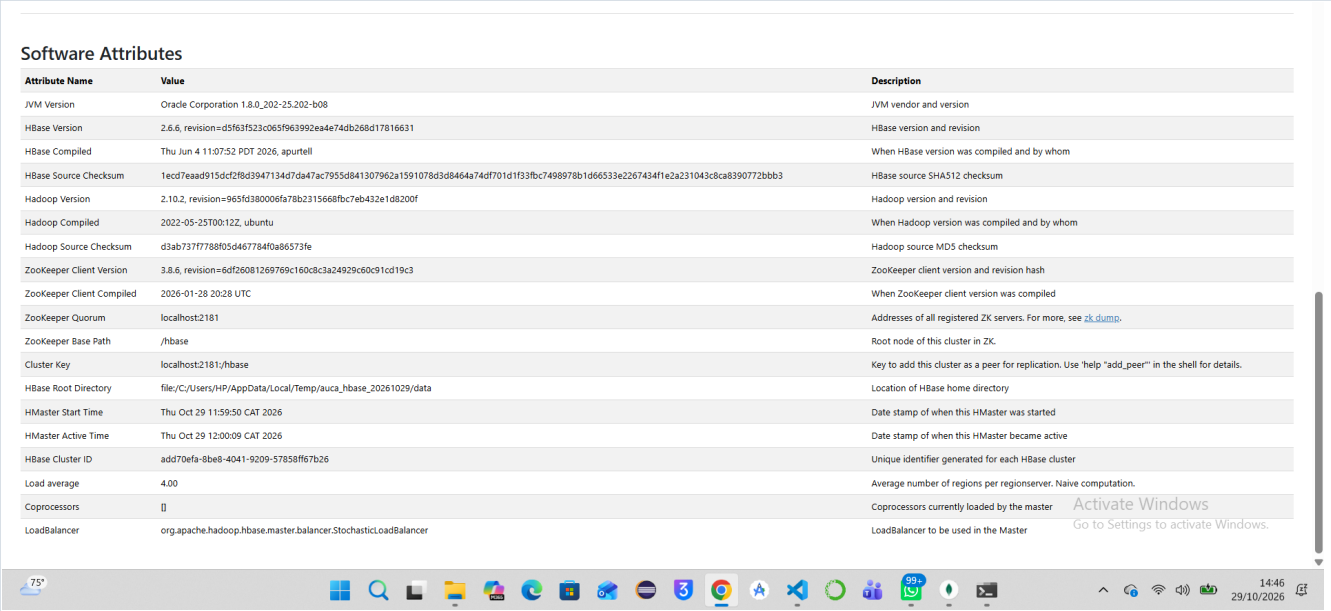


Figure 8. HBase Master software attributes (29/10/2026, 14:46).

Values read from the screenshot

Item	Observed
JVM	Oracle Corporation 1.8.0_202-25.202-b08
HBase	2.6.6, compiled Thu Jun 4 2026
Hadoop	2.10.2
ZooKeeper client	3.8.6
ZooKeeper quorum / base path	localhost:2181 ; /hbase
Cluster key	localhost:2181:/hbase
HBase root directory	file:/C:/Users/HP/AppData/Local/Temp/auca_hbase_20261029/data
HMaster start / active	Thu Oct 29 11:59:50 CAT 2026 / 12:00:09 CAT 2026
Load average	4.00
Coprocessors	[] (none)
Load balancer	StochasticLoadBalancer

Relevance to the report

These attributes document the exact runtime used for the HBase part of the project and support the reproduction notes: a local standalone HBase backed by the local file system, with a local ZooKeeper.

Reading notes

The root directory is a temporary folder, so the HBase data does not persist beyond a clean-up of that folder. This is acceptable for a demonstration but would not be for production. The load average of 4.00 is regions per region server, as the page states, not CPU load.

Comparison

MongoDB suits flexible, bounded business entities and multi-field lookups. HBase suits sparse, append-heavy event records and predictable row-key scans. Spark is used for broad joins and reductions rather than point retrieval. The databases complement one another; duplicating every record in both stores would add operational cost without improving the stated access patterns.

3. Spark processing and integration

The PySpark application reads multi-line JSON arrays, deduplicates primary IDs, parses event timestamps, explodes fulfilled transaction line items, and uses DataFrame joins and SQL. The co-purchase job self-joins distinct product IDs per order and counts order co-occurrence; a minimum support of two suppresses singletons. Cohort spend groups users by registration month and purchase month. Spark SQL joins line items to product categories and calculates revenue, units and order count.

Integrated business question

Which customer groups combine higher spending with stronger engagement, and where do sessions leave the purchase funnel? Spark joins user profiles, session counts and average durations, and fulfilled order totals. It calculates revenue per session and bands users into frequent (5+ orders), repeat (2-4), one-time (1), and no-purchase groups. The implemented store mode reads MongoDB profiles and fulfilled transaction aggregates and scans HBase user events directly through their native Python clients. It aggregates engagement by user, stages the extracts for Spark, joins on user_id, and writes metrics and customer segments. The verified run joined 1,000 profiles with 10,000 HBase sessions; it is distinct from the JSON-only batch path. CLV is represented by observed cumulative revenue, not a forecast.

The next two pages show the Spark UI during the full-scale run and the summary file it produced.

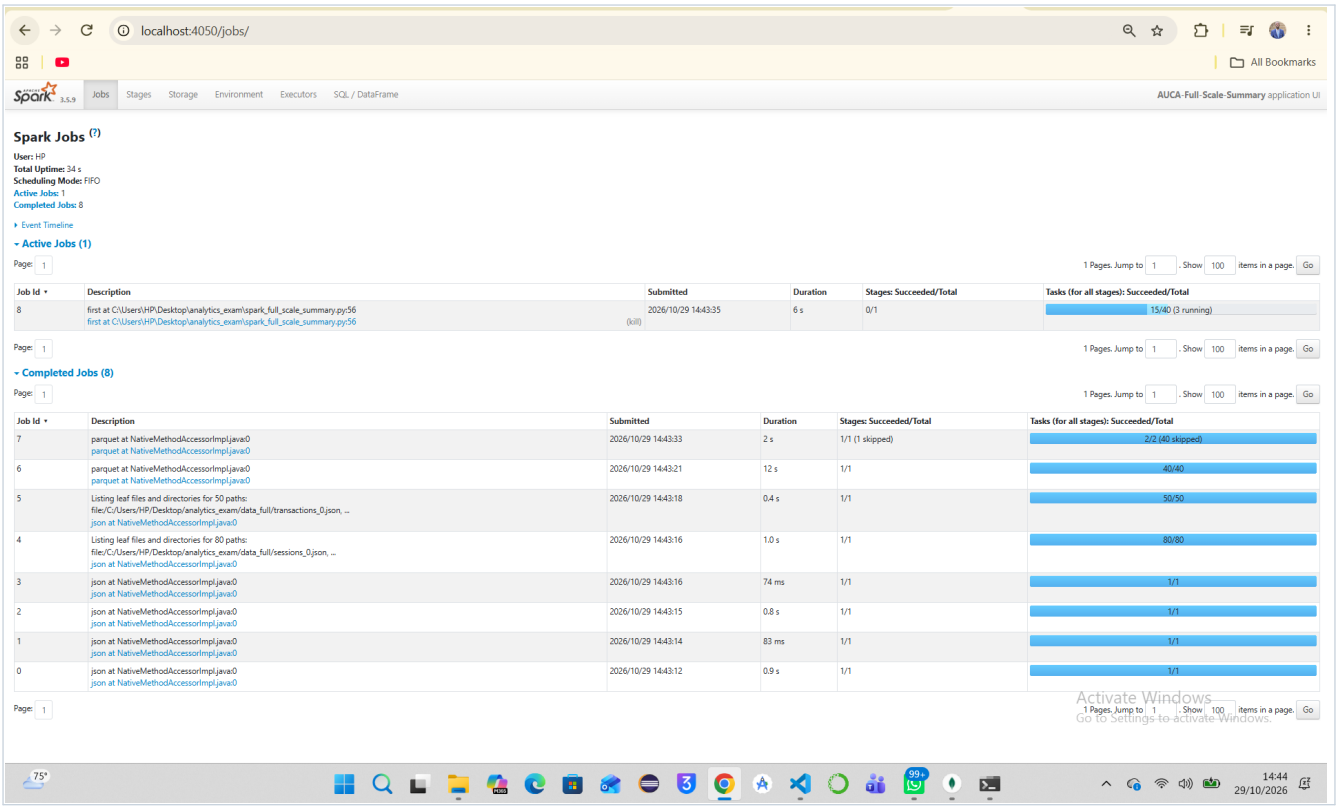


Figure 9. Spark UI, Jobs page, application AUCA-Full-Scale-Summary (localhost:4050, 29/10/2026, 14:44).

Values read from the screenshot

Item	Observed
Application / user	AUCA-Full-Scale-Summary; user HP; Spark 3.5.9 (as shown in the UI header)
Total uptime / scheduling	34 s; FIFO
Active jobs	1: Job 8, "first at ...\analytics_exam\spark_full_scale_summary.py:56", 15/40 tasks (3 running)
Completed jobs	8 (Jobs 0 to 7)
Jobs 0 to 3	json at NativeMethodAccessorImpl.java:0, 1/1 tasks each (JSON reads, likely schema inference)
Job 4	Listing leaf files for 80 paths: ...\data_full\sessions_0.json, ... (80/80 tasks, 1.0 s)
Job 5	Listing leaf files for 50 paths: ...\data_full\transactions_0.json, ... (50/50 tasks, 0.4 s)
Jobs 6 and 7	parquet at NativeMethodAccessorImpl.java:0; Job 6 took 12 s (40/40 tasks); Job 7 took 2 s

Relevance to the report

The UI shows the full-scale batch job reading the generated chunk files: 80 session paths and 50 transaction paths, exactly the file counts implied by the manifest. It evidences that Spark ran over the full files, as claimed in the scope table.

Reading notes

The capture was taken while Job 8 was still running, so it shows progress rather than a final state. The final numbers come from the summary file on the next page. The run uses local mode on one Windows machine, which is why this is not presented as a cluster benchmark.

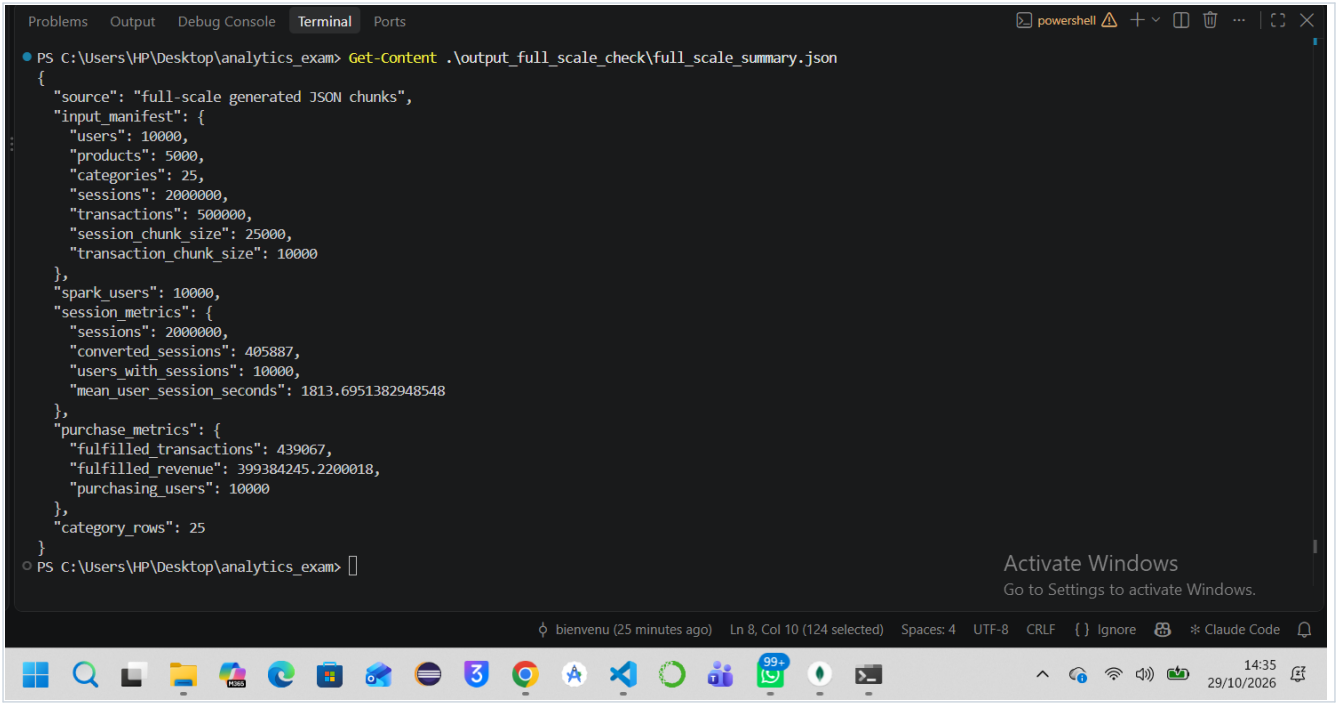


Figure 10. full_scale_summary.json printed in the terminal (29/10/2026, 14:35).

Values read from the screenshot

Item	Observed
Command	`Get-Content .\output_full_scale_check\full_scale_summary.json`
source	"full-scale generated JSON chunks"
input_manifest	users 10,000; products 5,000; categories 25; sessions 2,000,000; transactions 500,000; chunk sizes 25,000 and 10,000
spark_users	10,000
sessions / converted_sessions	2,000,000 / 405,887
users_with_sessions	10,000
mean_user_session_seconds	1813.6951382948548 (about 30.2 minutes)
fulfilled_transactions	439,067
fulfilled_revenue	399,384,245.2200018
purchasing_users	10,000
category_rows	25

Relevance to the report

These are the full-scale results quoted in Section 3 and in the Limitations: 405,887 converted sessions (20.3% of 2,000,000), 439,067 fulfilled transactions (87.8% of 500,000) and 399,384,245.22 in fulfilled revenue, an average of about 909.62 per fulfilled transaction.

Reading notes

Every one of the 10,000 users has sessions and at least one fulfilled purchase, so the full-scale data contains no no-purchase customers, unlike the 1,000-user sample where 115 users did not purchase. Both facts come from the synthetic generator, which is why the report treats the rankings as workflow demonstrations.

Assignment-derived findings

Measure	Observed in assignment-derived run	Interpretation
Sessions and transactions	10,000 sessions; 2,500 transactions	Generated assignment data at reduced scale.
Funnel stages	9,885 product-view sessions; 6,732 cart sessions; 2,057 converted	20.6% of sessions converted; 30.6% of cart sessions converted.
Purchase frequency	77 frequent, 560 repeat, 248 one-time, 115 no purchase	Purchase-frequency bands use fulfilled transaction history.
Revenue by category	cat_009 leads at 274,758.91; cat_012 next at 191,715.93	Synthetic rankings are useful for workflow demonstration, not market inference.

MongoDB ranked prod_00292 first by units, with 40 units across 19 orders and 12,685.20 in item revenue. MongoDB and Spark independently ranked cat_009 first at 274,758.91 item revenue. The fulfilled orders produced 3,472 distinct co-purchase pairs before the minimum support filter; the most frequent pair appeared in three orders. These are synthetic reduced-scale results, not validated recommendations. Full-scale Spark summaries processed 2,000,000 sessions and 500,000 transactions: 405,887 converted sessions (20.3%), 439,067 fulfilled transactions, and 399,384,245.22 in fulfilled revenue across 25 category rows. These full-scale measures are separate from the sample-based MongoDB/HBase integration.

4. Visual findings

All figures in this section are generated by spark_analytics.py from the assignment generator code at the reduced local scale described above. Values marked "approx." are read from the bar heights; all other values are quoted from the report text or the chart labels.

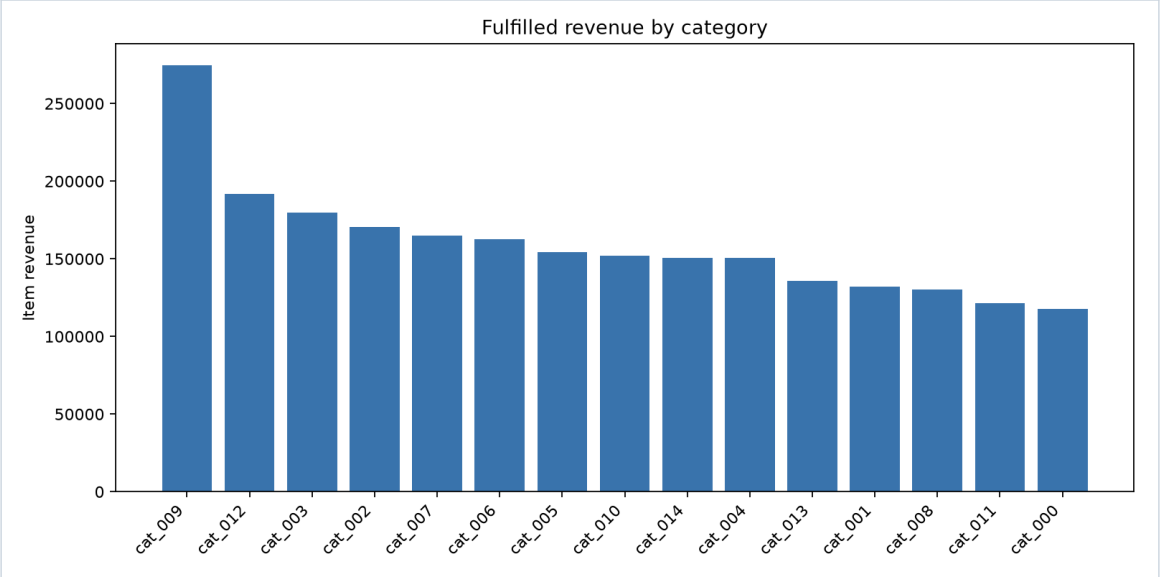


Figure 11. Fulfilled line-item revenue by category.

Values

Item	Value
Leader	cat_009, 274,758.91
Second	cat_012, 191,715.93
Next three (approx.)	cat_003 about 180,000; cat_002 about 170,000; cat_007 about 165,000
Lowest (approx.)	cat_000 about 117,000
Categories shown	15, matching the 15 category documents in MongoDB

Reading the chart

Revenue is concentrated in the first category, which earns about 1.4 times the second. After the top two the bars decline gradually, so no other category stands out. MongoDB and Spark gave the same leading category independently, which is the cross-check cited in Section 3. The dashboard (Figures 15 and 16) shows the same ranking for its own filtered window.

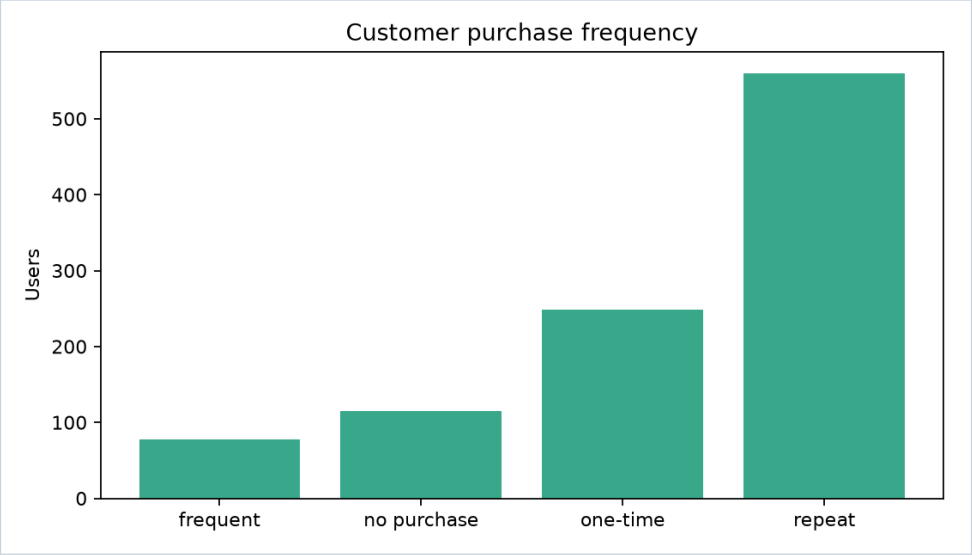


Figure 12. Customers grouped by purchase frequency.

Values

Item	Value
frequent (5+ orders)	77 users (7.7%)
repeat (2-4 orders)	560 users (56.0%)
one-time (1 order)	248 users (24.8%)
no purchase	115 users (11.5%)
Total	1,000 users

Reading the chart

More than half of the sampled users are repeat buyers, while only about one in thirteen reaches the frequent band. The 115 non-purchasers are the natural target for the funnel analysis in the next chart. The four groups add up to the 1,000 sampled profiles joined in the integration run.

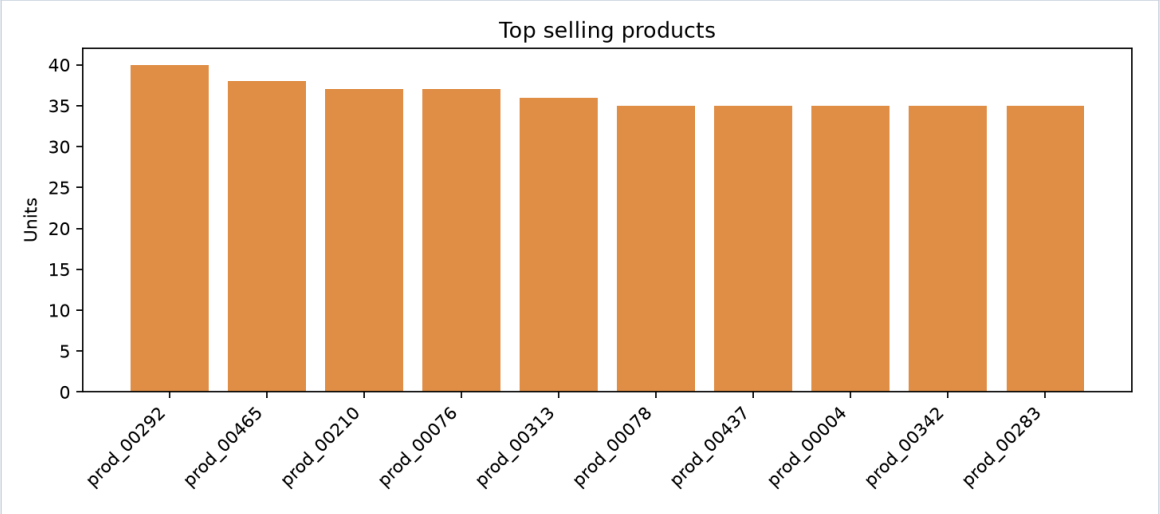


Figure 13. Top products ranked by units sold.

Values

Item	Value
prod_00292	40 units (12,685.20 item revenue, 19 orders)
prod_00465	38 units
prod_00210 and prod_00076	37 units each
prod_00313	36 units
prod_00078, 00437, 00004, 00342, 00283	35 units each

Reading the chart

The top ten products sit in a narrow band of 35 to 40 units, so ranking by units separates them only marginally. Ranking by revenue gives a different order: the dashboard's best-selling table (Figure 15) lists prod_00078 first at 18,068.40 with 35 units, because revenue depends on price as well as volume. Both views are correct for their own metric.

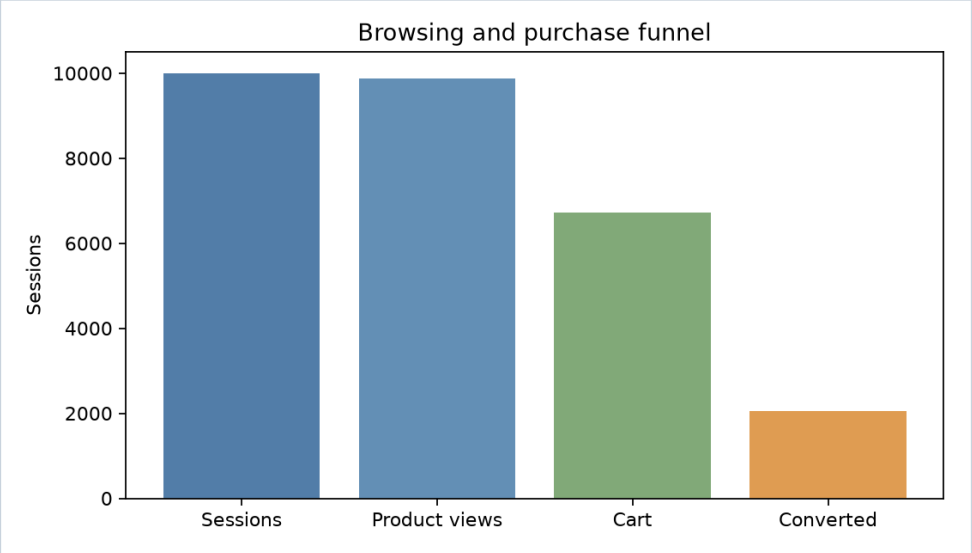


Figure 14. Sessions progressing through product view, cart and conversion.

Values

Item	Value
Sessions	10,000
Product views	9,885 (98.9% of sessions)
Cart	6,732 (67.3% of sessions)
Converted	2,057 (20.6% of sessions; 30.6% of cart sessions)
Largest drop	Product view to cart: 3,153 sessions lost (31.9% of product-view sessions)

Reading the chart

Nearly every session reaches a product page, so browsing is not the bottleneck. The sharpest absolute loss is between product view and cart, with a further steep drop from cart to conversion. At full scale the conversion rate is 20.3%, close to the sample's 20.6%, which suggests the sample is representative on this measure.

Simulated streaming

A separate Spark Structured Streaming job uses the rate source to generate synthetic page events, groups them into five-second event-time windows by page type and category, and writes each microbatch result. The verified 40-second run produced 10 microbatches and 346 windowed groups. This is a simulated stream, not a live production event source.

Parameter	Value
Source	Spark rate source (synthetic events)
Window	5 seconds, event time, grouped by page type and category
Run length / rate	40 seconds at 10 rows per second (from the reproduction command)
Result	10 microbatches; 346 windowed groups

The run length and rate are those of the documented reproduction command, which is a good guide to the order of magnitude: about 400 synthetic events in total. The 346 windowed groups are therefore the number of distinct (window, page type, category) combinations, not the number of events.

5. Interactive dashboard

The "Commerce intelligence" dashboard (AUCA Analytics Lab) presents the stored data through five tabs: Overview, Sales explorer, Customer 360, HBase activity and Data explorer. The sidebar shows live connections to MongoDB (mongodb://localhost:27017) and the HBase Thrift gateway (localhost:9090), with auto-refresh every 10 seconds and filters for transaction date range (31 Jul to 29 Oct 2026), order status (completed, shipped, delivered) and category.

Tab	Source	Purpose
Overview	MongoDB filtered results plus Spark full-scale summary	Headline KPIs, revenue trend and mix, best sellers, payment mix, full-scale batch strip
Sales explorer	MongoDB transactions and products	Revenue or unit trend, category performance, product ranking
Customer 360	MongoDB plus HBase	Sessions, fulfilled orders and revenue for one user, with recent HBase events
HBase activity	HBase user_events (live prefix scan)	Event count and page-type breakdown for one user
Data explorer	MongoDB (any collection)	Browse raw documents with a row limit and optional user filter

The screenshots on the next five pages were captured between 15:03 and 15:06 on 29/10/2026. Filters on the left of every capture are the same: all three statuses and the first categories.

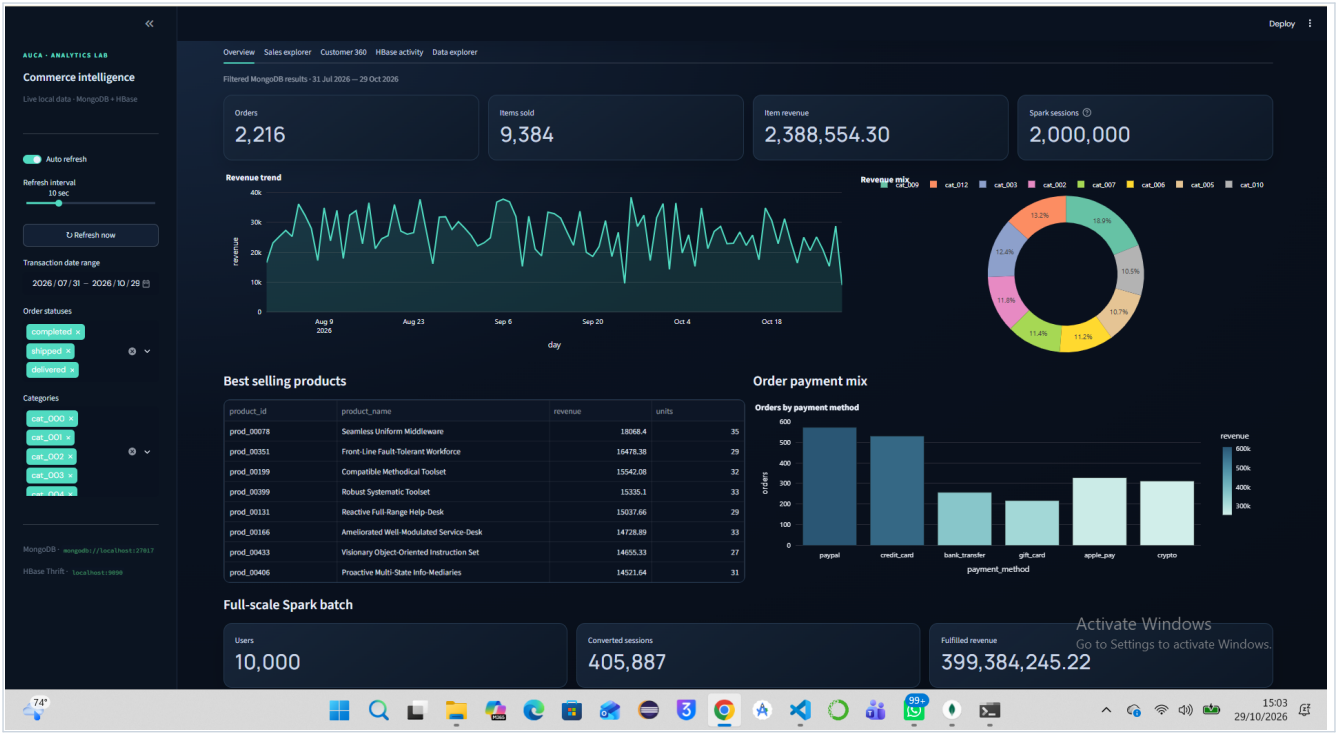


Figure 15. Dashboard, Overview tab (15:03).

Values read from the screenshot

Item	Observed
Filter caption	"Filtered MongoDB results: 31 Jul 2026 to 29 Oct 2026"
KPIs	Orders 2,216; Items sold 9,384; Item revenue 2,388,554.30; Spark sessions 2,000,000
Revenue mix (share)	cat_009 18.9%; cat_012 13.2%; cat_003 12.4%; cat_002 11.8%; cat_007 11.4%; cat_006 11.2%; cat_005 10.7%; cat_010 10.5%
Best selling products	prod_00078 Seamless Uniform Middleware 18,068.40 (35 units); prod_00351 16,478.38 (29); prod_00199 15,542.08 (32); prod_00399 15,335.10 (33)
Orders by payment method (approx.)	paypal about 580; credit_card about 520; apple_pay about 330; crypto about 310; bank_transfer about 250; gift_card about 215
Full-scale Spark batch strip	Users 10,000; Converted sessions 405,887; Fulfilled revenue 399,384,245.22

Relevance to the report

The Overview tab joins the two scales used in the project on one screen: MongoDB results for the filtered window above, and the full-scale Spark summary below. The strip values equal those in the summary file (Figure 10), and the leading category matches the category chart (Figure 11).

Reading notes

Derived from the KPIs: about 4.2 items per order, about 1,077.87 revenue per order and about 254.53 per item. The 2,216 orders are fewer than the 2,500 sample transactions because the dashboard counts only the selected statuses inside the date window, so the two numbers measure different things.

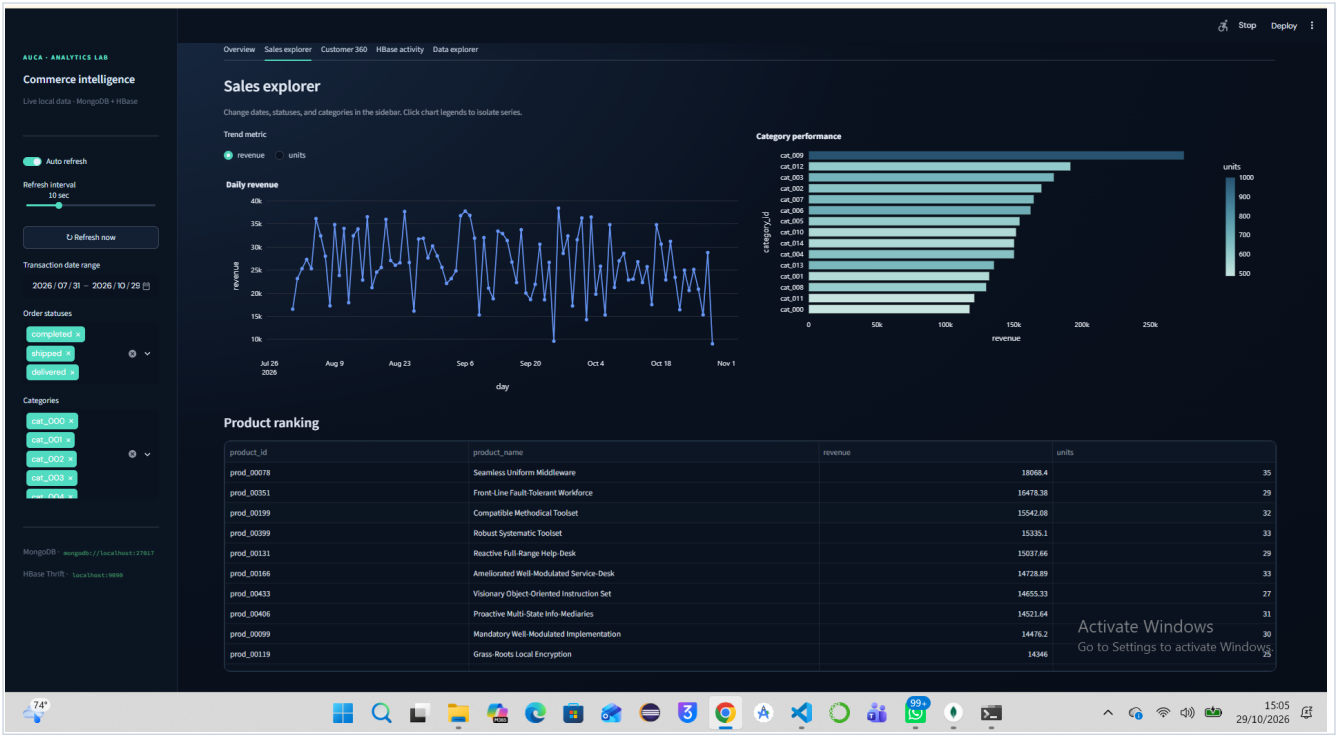


Figure 16. Dashboard, Sales explorer tab (15:05).

Values read from the screenshot

Item	Observed
Trend metric	Radio selector: revenue (selected) or units
Daily revenue	Typically 15k to 38k per day; lowest points near 9k to 10k
Series end	The last point is the lowest of the series, dropping to about 9k
Category performance	Horizontal bars by revenue, colour scale by units (500 to 1000); cat_009 about 270k at the top, cat_000 about 115k at the bottom
Product ranking	Same top products as the Overview, then prod_00433 14,655.33 (27 units), prod_00406 14,521.64 (31), prod_00099 14,476.20 (30), prod_00119 14,346 (25)

Relevance to the report

This tab lets the reader explore the same data as Figures 11 and 13 by date, status and category. The category ordering agrees with the static chart, which cross-validates the dashboard queries against the Spark output.

Reading notes

The final point of the daily series falls sharply. That may simply be a partial last day, so it should be checked before being read as a real decline. Values on this page are read from the chart and are approximate.

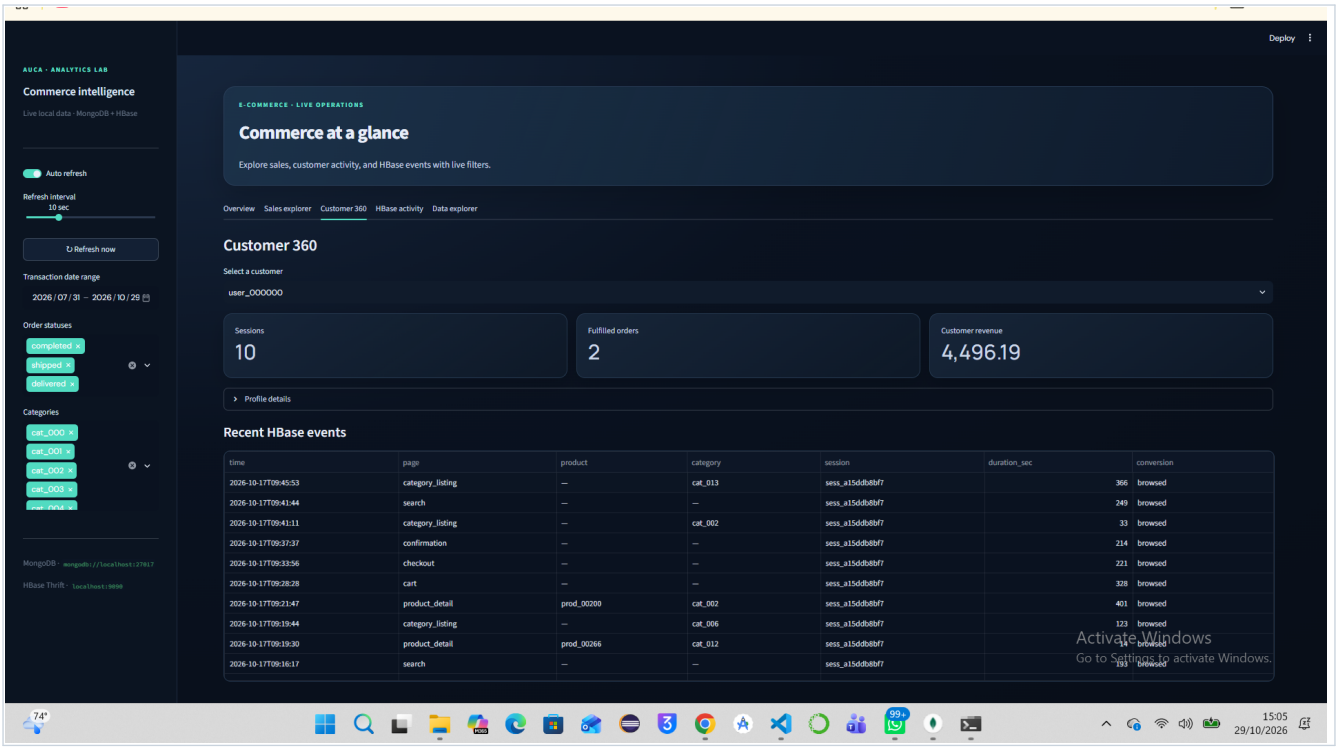


Figure 17. Dashboard, Customer 360 tab for user_000000 (15:05).

Values read from the screenshot

Item	Observed
Selected customer	user_000000
Sessions	10
Fulfilled orders	2
Customer revenue	4,496.19
Profile details	Collapsed expander
Recent HBase events	Newest first, from 2026-10-17T09:45:53 (category_listing) back to 2026-10-17T09:16:17 (search), all in session sess_a15ddb8bf7
Event columns	time, page, product, category, session, duration_sec, conversion
Conversion value in visible rows	"browsed" on every row

Relevance to the report

Customer 360 demonstrates the cross-store idea: revenue and order counts come from MongoDB transactions, and the event history comes from HBase through a row-key scan. It is the user-facing version of the integrated business question in Section 3.

Reading notes

The customer has two fulfilled orders, yet every visible event is labelled "browsed". This is consistent with the Limitations note that session-linked and standalone transactions must be distinguished: the orders are not necessarily tied to this session.

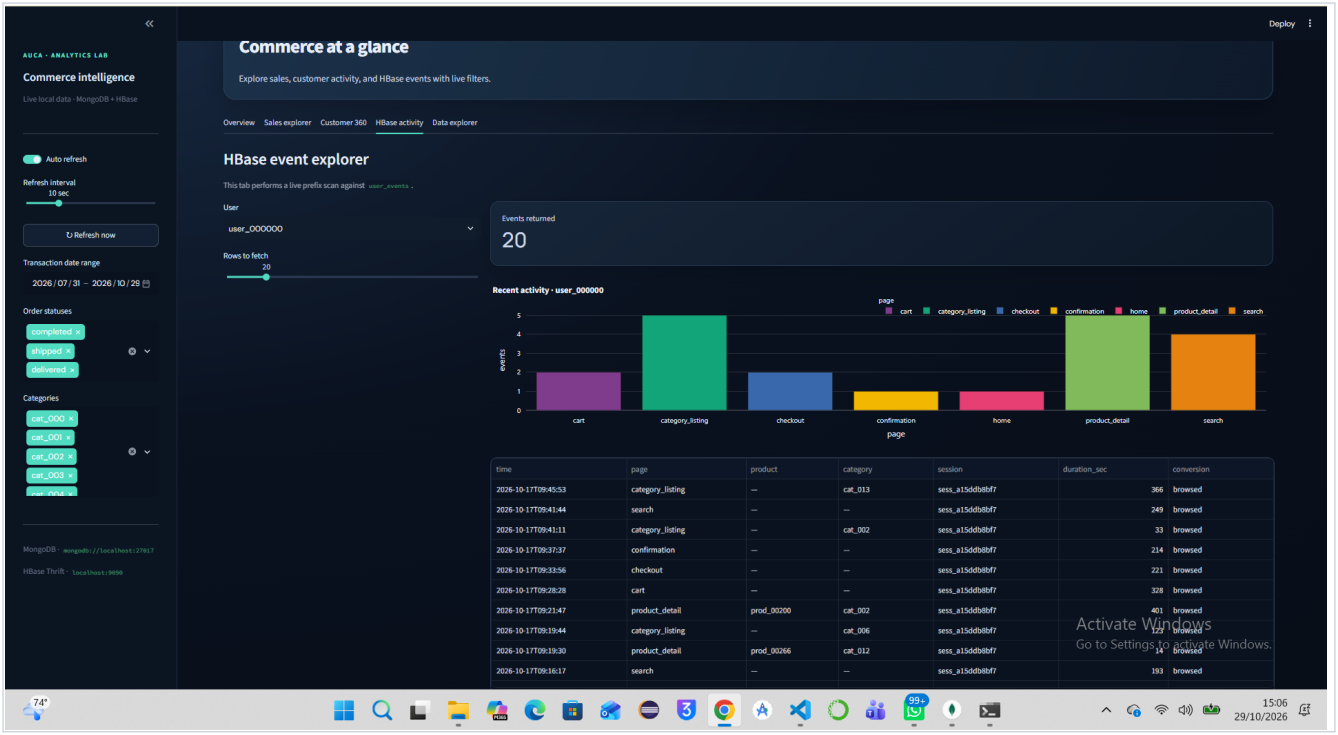


Figure 18. Dashboard, HBase activity tab for user_000000 (15:06).

Values read from the screenshot

Item	Observed
Query	Live prefix scan against user_events
User / rows to fetch	user_000000 / 20
Events returned	20
Events by page	cart 2; category_listing 5; checkout 2; confirmation 1; home 1; product_detail 5; search 4 (total 20)
Newest rows	2026-10-17T09:45:53 category_listing (366 s); 09:41:44 search (249 s); 09:41:11 category_listing (33 s); 09:37:37 confirmation (214 s)

Relevance to the report

This is the user-prefix scan from the HBase execution section, shown through the dashboard. The page counts add up to the 20 events returned, and the table is ordered newest first, which is the behaviour the reverse-epoch row key is designed to give.

Reading notes

Reading the table from the top, the journey runs in reverse: a confirmation page precedes the checkout and cart pages in the list, so in time order the user went cart, checkout, confirmation. The event times of this user are in October 2026, within the dashboard's date window.

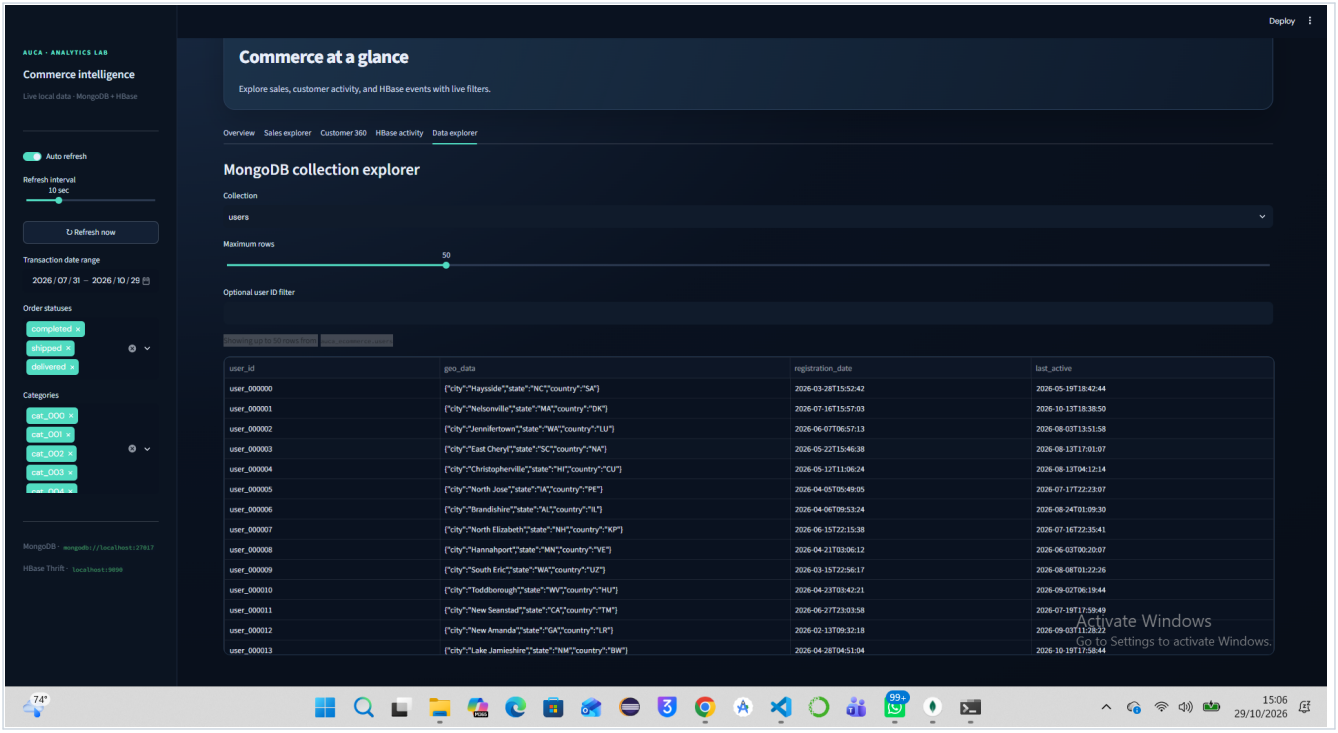


Figure 19. Dashboard, Data explorer tab, users collection (15:06).

Values read from the screenshot

Item	Observed
Collection / rows	users; maximum 50 rows
Source caption	"Showing up to 50 rows from auca_ecommerce.users"
Columns	user_id, geo_data, registration_date, last_active
user_000000	geo_data {"city":"Hayside","state":"NC","country":"SA"}; registered 2026-03-28T15:52:42; last active 2026-05-19T18:42:44
user_000001	{"city":"Nelsonville","state":"MA","country":"DK"}; registered 2026-07-16T15:57:03; last active 2026-10-13T18:38:50
user_000013	{"city":"Lake Jamieshire","state":"NM","country":"BW"}; registered 2026-04-28T04:51:04; last active 2026-10-19T17:58:44

Relevance to the report

The explorer shows the users documents behind Customer 360, with the nested geo fields described in the users collection design. Registration and last-active dates are all in 2026, consistent with the cohort definition by registration month.

Reading notes

City, state and country combinations are synthetic (for example a US-style state code with a non-US country code), which is a further reason why the findings in this report demonstrate the workflow only.

6. Scalability, limitations and reproduction

Scalability considerations

At full scale, use bulk writes, bounded batches and retry-safe identifiers for MongoDB; shard only after choosing shard keys aligned to common filters. HBase event keys should distribute writes while retaining user-level scan locality. Spark should partition large input by date or stable key, avoid collecting unbounded data to the driver, broadcast only genuinely small dimensions, and persist only reused intermediate frames. Parquet outputs reduce repeated JSON parsing. Track skew, shuffle volume, memory pressure and storage growth before tuning partitions.

Limitations

The full-scale JSON dataset and focused Spark summaries were run at the assignment generator defaults. The full-scale summary reports 2,000,000 sessions, 405,887 converted sessions, 439,067 fulfilled transactions and 399,384,245.22 in fulfilled revenue. MongoDB and HBase ingestion and the direct cross-store join use the smaller representative sample. This local run is not a distributed-cluster performance benchmark. The synthetic generator has no age or demographic age field, so purchase-frequency and registration cohorts replace age segmentation. Session-linked and standalone transactions should be distinguished when calculating funnel attribution. Current inventory values are snapshots rather than a historical stock ledger. MongoDB loading and both aggregation pipelines were executed locally. The HBase event and product/day loads were executed through local Thrift; 99,873 event rows and 24,881 product/day rows were verified, and a 20-row scan was run for user_000042. The direct Spark integration was executed against MongoDB and HBase. The streaming example was exercised with synthetic Spark rate source. The original notebook JSON generator remains unmodified.

Reproduction

Install the packages in requirements.txt and start local HBase with the project script. Generate assignment-derived data with `python run_assignment_generator.py --out data_assignment --sessions 10000 --transactions 2500`. To regenerate full data use `python run_assignment_generator.py --out data_full --users 10000 --products 5000 --categories 25 --sessions 2000000 --transactions 500000`. Run full-scale summaries with `spark-submit --master local[2] spark_full_scale_summary.py --data data_full --schema-sample data_assignment --out output_full_scale_summary`. For a quick smoke run use `python generate_sample_data.py --out data`. On Windows set `$env:PYSPARK_PYTHON=(Get-Command python).Source`, then run `spark-submit --master local[*] spark_analytics.py --data data_assignment --out output`. Run the direct MongoDB/HBase integration with `spark-submit --master local[2] spark_analytics.py --source-mode stores --out output`, and simulated streaming with `spark-submit --master local[2] spark_streaming_simulation.py --duration-seconds 40 --rows-per-second 10`. Create this PDF with `python build_report.py`. MongoDB loader/queries run with `python mongodb_analytics.py --data data_assignment`.

Conclusion

The project demonstrates MongoDB for bounded business entities, HBase for time-ordered sparse events and product/day metrics, and Spark for batch, cross-store and simulated stream processing. Full-scale Spark batch analysis was run on the generator defaults; the verified database loads and direct store join use the assignment-scale sample. The generator remains unchanged, with bounded JSON chunking handled by its runner. Synthetic findings demonstrate the workflow and are not market estimates.

Appendix A. Evidence timeline

All screenshots carry the system date 29/10/2026. The clock times below are taken from the Windows taskbar in each capture, which is why the taskbar was kept visible. They give the order in which the evidence was produced.

Time	Evidence	Figure
14:26	Full-scale dataset generation completed (2,000,000 sessions, 500,000 transactions)	Figure 1
14:28	dataset_manifest.json printed	Figure 2
14:35	full_scale_summary.json printed	Figure 10
14:36	run_hbase_local.ps1 output: 99,873 and 24,881 rows loaded; user_000042 scan	Figure 5
14:44	Spark UI of AUCA-Full-Scale-Summary (job 8 running)	Figure 9
14:46	HBase Master software attributes	Figure 8
14:47	HBase Master home and Region Visualizer	Figures 6 and 7
14:48	MongoDB Compass: categories collection	Figure 3
14:49	MongoDB Compass: products collection	Figure 4
15:03	Dashboard Overview	Figure 15
15:05	Dashboard Sales explorer and Customer 360	Figures 16 and 17
15:06	Dashboard HBase activity and Data explorer	Figures 18 and 19

One sequencing detail is worth knowing. The summary file (14:35) was produced before the Spark UI capture (14:44), so the UI shows a later run of the same application rather than the run that produced the file. Both read the same 80 and 50 chunk files, and the numbers in the file were not re-derived from the UI.

Appendix B. Environment and versions

Component	Value	Source
Operating system	Windows (user HP, host DESKTOP-CK0HHMD)	Taskbar and Master page
HBase	2.6.6, standalone, root dir on local disk	Software attributes
Hadoop libraries	2.10.2	Software attributes
ZooKeeper	Client 3.8.6; quorum localhost:2181; base path /hbase	Software attributes
JVM	Oracle 1.8.0_202	Software attributes
Spark	3.5.9 as shown in the UI header; local mode	Spark UI
MongoDB	mongodb://localhost:27017, inspected with MongoDB Compass	Dashboard sidebar, Compass
HBase Thrift	localhost:9090	Dashboard sidebar, loader output
HBase Master UI	localhost:16010	Browser address bar
Region server RPC	port 16020	Master page
Spark UI	localhost:4050	Browser address bar

Using non-default ports for the Spark UI (4050 rather than the usual 4040) is harmless but should be noted when reproducing the run, since the address in the browser will differ.

Appendix C. Data dictionary

Fields below are those visible in the screenshots or stated in the report. The report itself does not list every field of the generator output, so this dictionary is not claimed to be complete.

MongoDB

Collection	Field	Observed example / meaning
products	product_id	"prod_00000"; unique product key
products	name	"Synergistic Solution-Oriented Project"
products	category_id	"cat_011"; join key to categories
products	base_price	185.51 (number)
products	current_stock	631; stock snapshot
products	is_active	true
products	price_history	Array of bounded price records (1 to 3 entries seen)
products	creation_date	"2026-05-22T08:41:37"
categories	category_id	"cat_000" to "cat_014" in the sample
categories	name	Synthetic company-style name
categories	subcategories	Array of 4 or 5 entries
users	user_id	"user_000000"
users	geo_data	Nested object: city, state, country
users	registration_date / last_active	ISO-like timestamps, 2026
transactions	(design)	Order with embedded items array; fulfilled orders carry revenue and units
sessions	(design)	Session with nested page_views, device and cart

HBase

Table	Column / key part	Meaning
user_events	Row key	user_id | reverse_epoch_ms | session_id | event_index
user_events	e family	Page type, product, category, duration, event timestamp
user_events	s family	Session attributes
product_daily	Row key	product_id | yyyyMMdd
product_daily	m family	Product views, cart-add units, purchased units, revenue in cents

Dashboard event table

Column	Example
time	2026-10-17T09:45:53
page	category_listing, search, confirmation, checkout, cart, product_detail, home
product	prod_00200 on product_detail rows; empty otherwise
category	cat_013, cat_002, cat_006, cat_012
session	sess_a15ddb8bf7
duration_sec	366, 249, 33, 214, ...

Column	Example
conversion	browsed

Appendix D. Claim-to-evidence matrix

Each row lists a claim made in this report and the figure that evidences it. Claims marked "report text" rest on the written results of the analysis runs, for which this document contains no screenshot.

Claim	Evidence	Figure
Full dataset generated: 10,000 users, 5,000 products, 25 categories, 2,000,000 sessions, 500,000 transactions	Generator run; manifest; summary file	Figures 1, 2, 10
Spark processed all 2,000,000 sessions and 500,000 transactions	Spark UI reading 80 and 50 chunk files; summary file	Figures 9, 10
405,887 converted sessions; 439,067 fulfilled transactions; 399,384,245.22 revenue	Summary file; dashboard strip	Figures 10, 15
MongoDB loaded with products and categories	Compass collection views	Figures 3, 4
HBase 2.6.6 and Thrift running locally	Master UI; software attributes; loader output	Figures 6, 8, 5
user_events and product_daily tables created with the stated families	Master UI table descriptions	Figure 6
99,873 page-view rows and 24,881 product/day rows loaded	Loader output	Figure 5
20-row reverse-time scan for user_000042	Loader output	Figure 5
A user-prefix scan returns newest events first	Dashboard HBase activity table	Figure 18
cat_009 leads category revenue	Category chart; dashboard mix and bars	Figures 11, 15, 16
Purchase-frequency bands: 77, 560, 248, 115	Customer purchase frequency chart	Figure 12
Funnel: 9,885, 6,732, 2,057 of 10,000 sessions	Funnel chart	Figure 14
Top product by units: prod_00292, 40 units	Top selling products chart	Figure 13
Both MongoDB pipelines executed; 3,472 co-purchase pairs	Report text	none
Streaming run: 10 microbatches, 346 windowed groups	Report text	none
Direct MongoDB/HBase Spark join of 1,000 profiles with 10,000 sessions	Report text	none

Appendix E. Reconciliation of figures across sources

The same quantities appear in several places. The table shows that they agree and gives the ratios derived from them.

Full-scale batch

Quantity	Value	Where it appears
Sessions	2,000,000	Manifest, summary file, Spark UI chunk counts, dashboard KPI
Converted sessions	405,887	Summary file, dashboard strip, report text
Conversion rate	20.29%	Derived: 405,887 / 2,000,000 (report: 20.3%)
Fulfilled transactions	439,067	Summary file, report text
Fulfilled share of transactions	87.81%	Derived: 439,067 / 500,000
Fulfilled revenue	399,384,245.22	Summary file, dashboard strip, report text
Revenue per fulfilled transaction	about 909.62	Derived: 399,384,245.22 / 439,067
Mean user session time	1,813.70 s (30.2 min)	Summary file
Session chunks	80	Derived: 2,000,000 / 25,000; Spark UI Job 4
Transaction chunks	50	Derived: 500,000 / 10,000; Spark UI Job 5

Assignment-scale sample

Quantity	Value	Where it appears
Users / products / categories	1,000 / 500 / 15	Report text; Compass shows 500 products and 15 categories
Sessions / transactions	10,000 / 2,500	Report text
Converted sessions	2,057 (20.57%)	Report text; funnel chart
Sessions reaching cart	6,732 (67.32%)	Funnel chart
Cart-to-conversion rate	30.56%	Derived: 2,057 / 6,732 (report: 30.6%)
Purchase frequency groups	77 + 560 + 248 + 115 = 1,000	Purchase frequency chart
HBase event rows / sessions	99,873 / 10,000	Loader output; about 10 events per session
Events returned for one user	20	Loader output (user_000042), dashboard (user_000000)

Dashboard window (31 Jul to 29 Oct 2026, selected statuses)

Quantity	Value	Derivation
Orders	2,216	Dashboard KPI
Items sold	9,384	Dashboard KPI
Item revenue	2,388,554.30	Dashboard KPI
Items per order	about 4.23	9,384 / 2,216
Revenue per order	about 1,077.87	2,388,554.30 / 2,216
Revenue per item	about 254.53	2,388,554.30 / 9,384

No contradictions were found between the sources. The apparent difference between the dashboard's 2,216 orders and the sample's 2,500 transactions is explained by the status and date filters.

Appendix F. Glossary

Term	Meaning in this report
Assignment-scale sample	The reduced dataset held in MongoDB and HBase: 1,000 users, 500 products, 10,000 sessions, 2,500 transactions.
Full-scale dataset	The generator defaults: 10,000 users, 5,000 products, 25 categories, 2,000,000 sessions, 500,000 transactions. Used only by Spark batch.
Fulfilled	Order status counted as revenue (completed, shipped or delivered in the dashboard filter).
Converted session	A session that ended in a purchase.
Row key	HBase's single sorted index. Here user_id plus a reversed timestamp, so newer events sort first.
Reverse epoch	A large constant minus the event time, so that ascending key order is descending time order.
Column family	HBase storage group. user_events uses s (session) and e (event); product_daily uses m (metrics).
Thrift gateway	The HBase service on port 9090 that lets Python (HappyBase) read and write HBase.
Hotspotting	Uneven load on a few HBase regions caused by keys that cluster writes.
Aggregation pipeline	MongoDB's staged processing of documents ($match, $unwind, $group, $lookup and so on).
Co-purchase	Two products appearing in the same order; support is the number of orders containing both.
Cohort	A group of users sharing a registration month, followed through purchase months.
CLV	Customer lifetime value, represented here only by observed cumulative revenue.
Microbatch	One trigger of a Spark Structured Streaming query.
Windowed group	A distinct (five-second window, page type, category) combination in the streaming output.
Storefile	HBase's on-disk data file; "storefileSize" in the Region Visualizer is its total size.

Notes on the limits of this evidence

Screenshots show that a step ran and what it printed; they do not prove how it was computed. Results without a screenshot (the two MongoDB pipelines, the co-purchase count, the direct cross-store join and the streaming run) are stated from the analysis output and should be accompanied by their output files when the report is submitted. All data are synthetic and the dashboards read a local development environment, so none of the figures should be read as business performance.